

EXPERIMENT 4

Missionaries and Cannibals Problem

Question

Write a Python program to solve the Missionaries and Cannibals problem.

Aim

To write a Python program to solve the **Missionaries and Cannibals problem** using the **Breadth First Search (BFS)** algorithm.

Algorithm

1. Start with 3 missionaries and 3 cannibals on the left side of the river.
2. The boat starts on the left side.
3. Generate all possible combinations of missionaries and cannibals that can travel in the boat.
4. Check whether the new state is safe.
5. A state is safe when missionaries are not outnumbered by cannibals.
6. Store the safe and unvisited states in a queue.
7. Continue until all missionaries and cannibals reach the right side.
8. Display the solution path.

Code

python exp 4.py - C:/Users/royal/python exp 4.py (3.13.14)

File Edit Format Run Options Window Help

```
from collections import deque

def safe(m, c):
    if m < 0 or c < 0 or m > 3 or c > 3:
        return False

    if m > 0 and m < c:
        return False

    mr = 3 - m
    cr = 3 - c

    if mr > 0 and mr < cr:
        return False

    return True

def solve():
    queue = deque()
    queue.append((3, 3, 1, []))

    visited = set()
    visited.add((3, 3, 1))

    moves = [(1,0), (2,0), (0,1), (0,2), (1,1)]

    while queue:
        m, c, boat, path = queue.popleft()

        if m == 0 and c == 0:
            for state in path + [(m, c, boat)]:
                print(state)
            return

        for dm, dc in moves:
            if boat == 1:
                nm = m - dm
                nc = c - dc
                nb = 0
            else:
                nm = m + dm
                nc = c + dc
                nb = 1

            state = (nm, nc, nb)

            if safe(nm, nc) and state not in visited:
                visited.add(state)
                queue.append((nm, nc, nb, path + [(m, c, boat)]))

solve()
```

Output

```
===== RESTART: C:/Users/royal/python exp 4.py ==  
(3, 3, 1)  
(3, 1, 0)  
(3, 2, 1)  
(3, 0, 0)  
(3, 1, 1)  
(1, 1, 0)  
(2, 2, 1)  
(0, 2, 0)  
(0, 3, 1)  
(0, 1, 0)  
(1, 1, 1)  
(0, 0, 0)  
|
```

Result

Thus, the **Missionaries and Cannibals** problem was **successfully solved using the Breadth First Search algorithm**, and all missionaries and cannibals were safely transported to the other side.